

Adapter Design Pattern

In this activity, I learned how the Adapter Design Pattern solves the problem of incompatible file-writing libraries. The attendance application used different methods for exporting records to CSV, JSON, and text files. The original implementation used format-based conditional statements, which made the `ReportExporter` dependent on every file format. This would require modifying the existing code whenever a new format, such as XML, was added.

To fix this problem, I created a common `FileWriter` interface and used adapter classes to connect the existing libraries to that interface. I completed the `TextFileAdapter` by looping through the records, converting each row into a text line, and checking whether each line was written successfully. I also completed the `ReportExporter` so that it delegates the writing process to the selected adapter. For extensibility, I added an additional adapter that allows another file format to be used without changing the exporter.

Through this activity, I understood how the Adapter Pattern allows incompatible classes to work together through a common interface. The adapter handles the differences between the existing library and the interface expected by the client. I also learned that this approach follows the Open-Closed Principle because the system can support new file formats by adding a new adapter instead of modifying the existing exporter. This makes the code more organized, reusable, and easier to maintain.